

AMD GPU Deep Learning Setup

A complete guide to configuring WSL2, ROCm, and PyTorch on Windows

Overview: Native AMD GPU support for PyTorch is optimized for Linux environments. On Windows systems, the most robust way to leverage an AMD GPU for AI workloads is by utilizing Windows Subsystem for Linux (WSL2), which natively supports hardware passthrough.

Part 1: Installing Windows Subsystem for Linux (WSL2)

1 Open an Administrator Terminal

Right-click your Windows Start button and select **Terminal (Admin)**, **Windows PowerShell (Admin)**, or **Command Prompt (Admin)**. Accept the User Account Control prompt if it appears.

2 Execute the Install Command

Type the following command and press Enter. This enables required Windows features, downloads the WSL kernel, and installs Ubuntu as the default Linux distribution.

```
wsl --install
```

3 Restart Your Computer

Once the command completes, a system reboot is strictly required to apply the virtual machine platform changes. Save your work and restart Windows.

4 Set up Linux Credentials

After rebooting, a Linux terminal window will open automatically (if not, search for "Ubuntu" in your Start menu). Wait for the initial setup to finish.

You will be prompted to create a **UNIX username** and a **password**. These are completely separate from your Windows credentials. When typing your password, characters will remain invisible—this is standard Linux security behavior. Press Enter to confirm.

Part 2: Installing AMD ROCm in WSL2

Windows automatically passes your AMD GPU through to WSL2, meaning you do not install standard Windows display drivers inside Ubuntu. Instead, we must install the specific ROCm toolkits that allow Linux libraries to compute on the passed-through GPU. Ensure all following commands are run inside your Ubuntu terminal.

1 Update Ubuntu Packages

Ensure your new Linux environment is up-to-date with the latest system packages.

```
sudo apt update && sudo apt upgrade -y
```

2 Download the AMD Driver Installer

Fetch the specific `amdgpu-install` package from the official AMD repository.

```
wget https://repo.radeon.com/amdgpu-install/6.2.3/ubuntu/jammy/amdgpu-install_6.2.60203-1_all.deb
```

3 Install the Package

Install the downloaded `.deb` package to register AMD's repositories with your system.

```
sudo apt install ./amdgpu-install_6.2.60203-1_all.deb -y
```

4 Install ROCm Specifically for WSL

Use the tool to install the ROCm components. The `--no-dkms` flag is absolutely critical here, as WSL manages its own kernel and DKMS compilation will fail. This step may take several minutes.

```
amdgpu-install -y --usecase=wsl,rocm --no-dkms
```

Part 3: Installing ROCm PyTorch

5 Install Python's Package Manager

Ubuntu includes Python by default, but you need pip to install PyTorch.

```
sudo apt install python3-pip -y
```

6 Install PyTorch Wheels for ROCm

Install the specific PyTorch wheels built for AMD ROCm (6.1 compatibility). By pointing the `--index-url` to the ROCm endpoint, you ensure pip doesn't download the default CPU or Nvidia CUDA versions.

```
pip3 install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/rocm6.1
```

7 Verify Hardware Acceleration

Test that PyTorch successfully sees and can communicate with your AMD GPU. Run this one-line Python script in your terminal:

```
python3 -c "import torch; print(f'GPU Available:  
{torch.cuda.is_available()}'); print(f'Device Name:  
{torch.cuda.get_device_name(0)}' if torch.cuda.is_available() else 'No GPU')"
```

If successful, the output will read **GPU Available: True** followed by the specific model name of your AMD Radeon graphics card.

Important Conceptual Note Regarding "CUDA"

You may notice that the verification script uses `torch.cuda.is_available()` despite using an AMD GPU. ROCm acts as a seamless translation layer (via HIP). In PyTorch code, the device target remains "cuda".

ROCm intercepts Python commands meant for NVIDIA hardware and natively maps them to your AMD hardware. You **do not** need to alter standard `model.to("cuda")` calls in external GitHub repositories.